

TP 1 — Détection de fake news avec un modèle de langage pré-entraîné (PLM)

PLM (BERT / CamemBERT / DistilBERT) + *fine-tuning* — comparaison de 3 approches

1. Objectifs

- Utiliser un PLM (modèle de langage pré-entraîné) pour **classer un texte comme fake ou real (fausse ou vraie information)**.
- Comparer 3 approches : (A) baseline classique, (B) PLM gelé utilisé comme extracteur de features.
- **Optionnel** : (C) PLM fine-tuné
- Mesurer rigoureusement (accuracy, F1) et JUSTIFIER le choix du modèle pour une mise en production.

3. Environnement

- Python 3.10+ ; bibliothèques : transformers, datasets, evaluate, scikit-learn, torch (voir requirements.txt).
- Docker, UV, GIT etc..

4. Jeu de données — choisir UNE option

Toutes sont binaires (positif / négatif) sauf indication. Ne pas mélanger : choisir une seule source et s'y tenir.

Option	Langue	Accès	Note
A — Fake and Real News (Kaggle, ISOT)	Anglais	clmentbisaillon/fake-and-real-news-dataset (2 CSV : Fake.csv, True.csv)	~44k articles complets, binaire propre. Piège : le style/la source trahit la classe → tester la généralisation.
B — LIAR (PolitiFact)	Anglais	load_dataset("liar")	~12,8k courtes déclarations politiques, 6 classes (à binariser : vrai vs faux). Plus difficile, plus réaliste (pas de style « article »).
C — WELFake	Anglais	davanstrien/WELFake (ou CSV Kaggle)	~72k articles (fusion de 4 sources), binaire. Volume élevé → bon pour comparer PLM gelé vs fine-tuné.

5. Déroulé

Partie A — Données & baseline

- Charger le dataset, **explorer** (taille, équilibre des classes), **prétraiter**, séparer train/validation/test.
- Baseline classique : TF-IDF + régression logistique. Noter accuracy et F1.

Partie B — PLM gelé (extracteur de features)

- Charger un PLM gelé (bert-base / distilbert / camembert selon la langue), extraire l'embedding.
- Entraîner un classifieur sur ces embeddings. Comparer à la baseline.

Partie C (**optionnel**) — Fine-tuning du PLM

- Fine-tuner le PLM sur la tâche (Trainer de Hugging Face). Surveiller le sur-apprentissage.

Partie D — Comparaison & décision

- Tableau comparatif (accuracy, F1, temps, coût), matrice de confusion, analyse des erreurs.

Bonus (Tier-2)

- Comparer à un LLM bien prompté (zero/few-shot via API) : **le fine-tuning valait-il le coup ?**

6. Questions (à traiter dans le compte-rendu)

1. Les classes sont-elles équilibrées ? Quel impact sur le choix de la métrique (accuracy vs F1) ?
2. Quel score obtient la baseline TF-IDF + régression logistique ?
3. PLM gelé vs baseline : quel gain ? Pourquoi les embeddings contextuels aident-ils ?
4. **Fine-tuning vs PLM gelé : quel gain ? À quel coût (temps, GPU) ?**
5. Matrice de confusion : quelles classes se confondent ? Donnez 2 exemples d'erreurs et une hypothèse.
6. Quel modèle choisiriez-vous pour la production, et pourquoi (qualité, coût, latence) ?
7. Comment avez-vous évité le sur-apprentissage et la fuite test-in-train ?
8. **Bonus** : un LLM bien prompté fait-il mieux — ou moins cher — que votre PLM fine-tuné ?

7. Livrables

- Un dépôt (notebook + code exécutable) et le compte-rendu ci-dessous, rempli.
- Le code doit être reproductible : requirements.txt, seed fixée, instructions d'exécution etc..

COMPTE-RENDU — TP 1

À remplir et à rendre

Identité

Nom(s) et prénom(s) : SAKHI SARA Filière / groupe : Master big data et internet d'objets

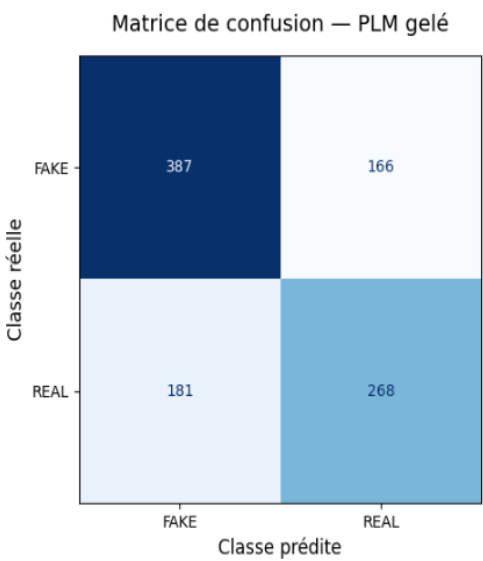
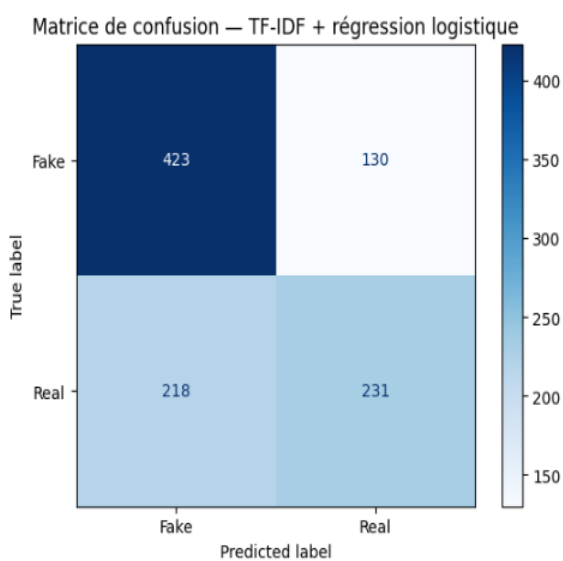
Date : 24/09/2026 Option de dataset choisie (A / B / C) : B

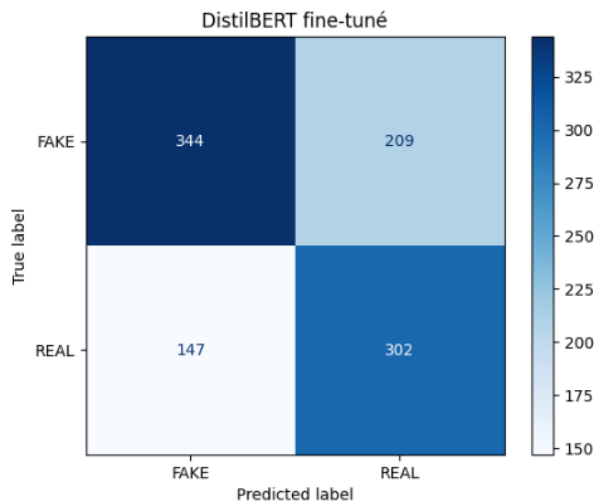
Modèle PLM utilisé : distilbert/distilbert-base-uncased

Résultats

Approche	Accuracy	F1	Temps d'entraînement	Coût (GPU / €)
A — Baseline (TF-IDF + LR)	0.6527	0.6395	0.0391 s	0 CPU
B — PLM gelé (features)	0.6537	0.6487	4.5960 s	CPU + GPU Colab gratuit
C — PLM fine-tuné (optionel)	0.6447	0.6441	238.5015 s	GPU colab gratuit
Bonus — LLM prompté Sur 100 TEXTES	0.75	0.7457	Déjà entrainer	modèle utilisé sans entraînement .

Matrice de confusion (coller une capture) :





Réponses aux questions

Q1. Les classes sont-elles équilibrées ? Quel impact sur le choix de la métrique (accuracy vs F1) ?

Les classes sont relativement équilibrées, malgré une légère majorité de la classe Fake : 55,23 % contre 44,77 % de Real dans le train, et 55,19 % contre 44,81 % dans le test. L'accuracy reste donc pertinente pour mesurer la performance globale. Nous utiliserons également le F1-score macro, qui calcule le F1-score de chaque classe séparément en leur accordant la même importance. Contrairement au F1-micro, qui agrège toutes les prédictions, et au F1-weighted, qui pondère chaque classe selon son effectif, le F1-macro permet de vérifier que le modèle est performant aussi bien sur Fake que sur Real, malgré leur léger déséquilibre.

Q2. Quel score obtient la baseline TF-IDF + régression logistique ?

TF-IDF + régression logistique_score=0.6395

Q3. PLM gelé vs baseline : quel gain ? Pourquoi les embeddings contextuels aident-ils ?

Avec TF-IDF + régression logistique, j'ai obtenu une accuracy de **65,27 %** et un F1-macro de **63,95 %**. Avec DistilBERT gelé + régression logistique, j'ai obtenu **65,37 %** d'accuracy et **64,87 %** de F1-macro. Le gain est donc de **0,10 point d'accuracy** et de **0,92 point de F1-macro**.

Les embeddings contextuels représentent les mots en tenant compte des autres mots de la phrase. Ils peuvent ainsi mieux représenter le sens qu'une représentation TF-IDF. Dans mon expérience, l'amélioration reste faible : comprendre une déclaration ne suffit pas toujours à vérifier si elle est vraie.

Q4. Fine-tuning vs PLM gelé : quel gain ? À quel coût (temps, GPU) ?

Dans mon expérience, le **fine-tuning n'améliore pas les résultats du PLM gelé**. L'accuracy passe de **65,37 %** à **64,47 %**, soit une baisse de **0,90 point**, et le F1-macro passe de **64,87 %** à **64,41 %**, soit une baisse de **0,46 point**. Le temps indiqué passe de **4,60 secondes** pour le PLM gelé, utilisant CPU et GPU, à **238,50 secondes** pour le fine-tuning sur GPU. Les ressources gratuites de Colab ont permis un **coût financier nul**. Le fine-tuning est donc ici plus coûteux en temps de calcul, sans gain de performance.

Q5. Matrice de confusion : quelles classes se confondent ? Donnez 2 exemples d'erreurs et une hypothèse.

Pour DistilBERT gelé, **166 textes Fake sont prédits Real** et **181 textes Real sont prédits Fake**. Les erreurs existent donc dans les deux sens, avec légèrement plus de textes Real classés comme Fake.

PHRASE 1 : « Wisconsin is on pace to double the number of layoffs this year. » La vraie classe est **Fake**, mais le modèle prédit **Real**. Mon hypothèse est que la formulation paraît plausible, alors que sa vérification nécessite des statistiques sur les licenciements et une période de référence, absentes du texte.

PHRASE 2 : « Building a wall on the U.S.-Mexico border will take literally years. » La vraie classe est **Real**, mais le modèle prédit **Fake**. Mon hypothèse est que le modèle s'appuie sur des associations de mots liées au sujet politique, sans disposer des informations permettant d'évaluer la durée réelle du chantier.

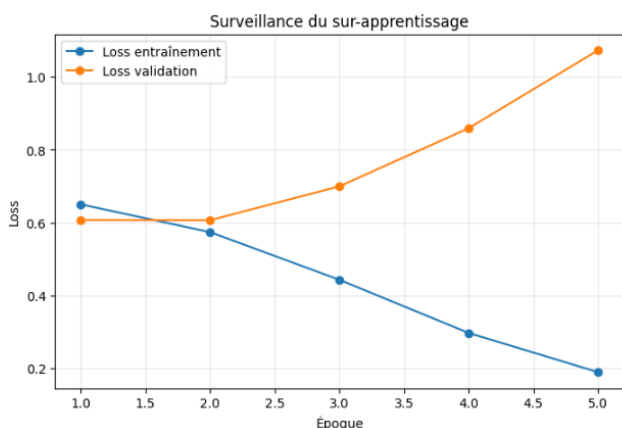
Q6. Quel modèle choisiriez-vous pour la production, et pourquoi (qualité, coût, latence) ?

Je choisirais **TF-IDF + régression logistique** pour un premier prototype, car ses performances sont proches de celles de DistilBERT, avec un coût de calcul plus faible et sans GPU. Son accuracy est de **65,27 %**, contre **65,37 %** pour le PLM gelé.

Sur un lot de 100 textes, le temps moyen mesuré est de **0,0714 ms par texte** pour la baseline, contre **1,4165 ms** pour le PLM gelé et **0,4417 ms** pour le modèle fine-tuné. Ces valeurs sont des moyennes sur un lot, et non des

latences de requêtes individuelles. Les performances restent insuffisantes pour utiliser le modèle comme un vérificateur autonome de faits.

Q7. Comment avez-vous évité le sur-apprentissage et la fuite test-in-train ?



Pour limiter le sur-apprentissage du fine-tuning, j'ai suivi les losses d'entraînement et de validation, utilisé une régularisation avec `weight_decay=0.01` et un arrêt anticipé après deux époques sans amélioration du F1-macro de validation. Le meilleur checkpoint, obtenu à l'époque 3, a été restauré.

Q8. Bonus : un LLM bien prompté fait-il mieux — ou moins cher — que votre PLM fine-tuné ?

J'ai testé Gemini 3.5 Flash-Lite et mon DistilBERT fine-tuné sur les **mêmes 100 textes**. J'ai utilisé une seed de 42 pour pouvoir retrouver exactement les mêmes textes.

Gemini a correctement classé **75 textes sur 100**, contre **67 pour DistilBERT**. Son F1-macro est aussi meilleur : **0,7457 contre 0,6673**. Gemini a reçu uniquement une consigne et le texte à classer, sans exemple corrigé : c'est le **zero-shot**.

Sur ce test, Gemini donne donc de meilleurs résultats sans que je doive l'entraîner. Il prend en moyenne **0,82 seconde par appel**. DistilBERT peut cependant fonctionner directement sur un ordinateur, sans utiliser une API. Je dois vérifier le coût des appels Gemini avant de dire lequel est moins cher.

Ces résultats concernent seulement 100 textes : ils ne prouvent pas que Gemini sera toujours meilleur. Il est aussi possible que Gemini ait déjà rencontré certaines déclarations de LIAR pendant son entraînement, car ce dataset est public.

Décision & justification

Modèle retenu pour la production et pourquoi (qualité / coût / latence, chiffres à l'appui) :

Je retiens **TF-IDF + régression logistique** pour la production, car il offre un bon compromis entre qualité et coût : **65,27 % d'accuracy**, **63,95 % de F1-macro** et **0,0391 s d'entraînement sur CPU**, sans coût GPU. Le PLM gelé apporte seulement **0,92 point de F1-macro supplémentaire**, avec un calcul des embeddings plus coûteux. Le fine-tuning demande **238,50 s** sans dépasser le PLM gelé. Le LLM obtient de meilleurs scores, mais son évaluation sur **100 textes seulement** ne permet pas une comparaison directe. Enfin, TF-IDF + LR convient à une prédiction légère sur CPU ; sa **latence d'inférence reste à mesurer**, les temps du tableau concernant l'entraînement.

Limites & pistes d'amélioration

Les déclarations de LIAR sont courtes et leur vérification nécessite parfois des informations externes. Mon modèle utilise le texte sans consulter de sources fiables pour vérifier les faits. La binarisation simplifie également des catégories nuancées, notamment `barely-true` et `mostly-true`.

J'ai essayé d'enrichir les données en ajoutant d'autres colonnes que `statement`, comme le sujet, le contexte, la profession, l'État et l'affiliation politique de l'intervenant, ainsi que les compteurs de ses déclarations antérieures. Cette approche a apporté une amélioration modeste, mais n'a pas permis d'obtenir un F1-macro élevé.

J'ai également testé une représentation TF-IDF combinant des n-grammes de mots et de caractères. Avec `GridSearchCV`, j'ai comparé plusieurs valeurs du paramètre `C` et l'utilisation ou non de `class_weight="balanced"`, en sélectionnant les paramètres sur la validation. Ces essais n'ont pas apporté l'amélioration d'accuracy espérée.

Ces résultats suggèrent que modifier les paramètres ou ajouter des métadonnées ne suffit pas toujours à distinguer une déclaration vraie d'une déclaration fausse. Une piste serait d'intégrer des preuves externes,

comme des statistiques officielles ou des documents fiables, afin de confronter les déclarations à des faits. Il faudrait aussi vérifier que les métadonnées utilisées sont disponibles au moment de la prédiction et ne révèlent pas indirectement la réponse.

Enfin, répéter les expériences avec plusieurs seeds permettrait de vérifier la stabilité des résultats. Une évaluation sur une autre source aiderait à mesurer la capacité du modèle à généraliser.